SENTINEL-STATE

Enterprise Architecture for Multi-State Medicaid Data

Component Interaction, Data Flow & Technical Design Reference

Version 2.0 | Includes System Architecture & Sequence Flow Diagrams | Classification: Confidential

How to Read This Document

This document is structured for progressive reading. Start with the Executive Summary for a high-level mental model. Study Figure 1 (System Architecture) to understand how all components connect spatially. Then follow Figure 2 (Sequence Flow) to understand the step-by-step runtime behavior. Deep-dive sections on each agent, the data lifecycle, and the HITL workflow follow for reference.

1. Executive Summary

Sentinel-State is an agentic, LLM-powered data orchestration platform built to solve one of the most persistent challenges in U.S. healthcare operations: the fragmented, inconsistent, and constantly-changing landscape of Medicaid fee schedules across 50 states.

Each state Medicaid program publishes reimbursement rate schedules in its own format — different file types, column naming conventions, portal structures, and update frequencies. Manually collecting, normalizing, and auditing these files is expensive, error-prone, and creates stale data that undermines reimbursement modeling and financial forecasting.

Sentinel-State eliminates this burden through a swarm of four specialized AI agents — Navigator, Extractor, Analyst, and Archivist — orchestrated by LangGraph and powered by Amazon Bedrock (Claude 3.5 Sonnet). The platform ingests raw documents automatically, normalizes them against a canonical schema, and persists validated data through a Bronze/Silver/Gold data lake pattern into Amazon Aurora for real-time querying.

Core Design Principles

1. Automation First, Humans as Guardrails — the pipeline runs fully autonomously by default. Humans are engaged only when the Analyst Agent's confidence in a column mapping falls below 85%, ensuring quality without sacrificing speed.2. Full Audit Lineage — every data point in the Gold layer is traceable back to the exact raw file in Bronze, including the source URL, ingestion timestamp, and agent version.3. HIPAA-Ready Security — KMS encryption at rest, TLS in transit, Least Privilege IAM per agent, and BAA-eligible AWS services throughout.4. Continuous Learning — human corrections are stored in Mapping Memory and reused in future runs, reducing HITL trigger frequency over time.

System Architecture Overview

Read this diagram before proceeding — it establishes the spatial layout of all components discussed in this document.

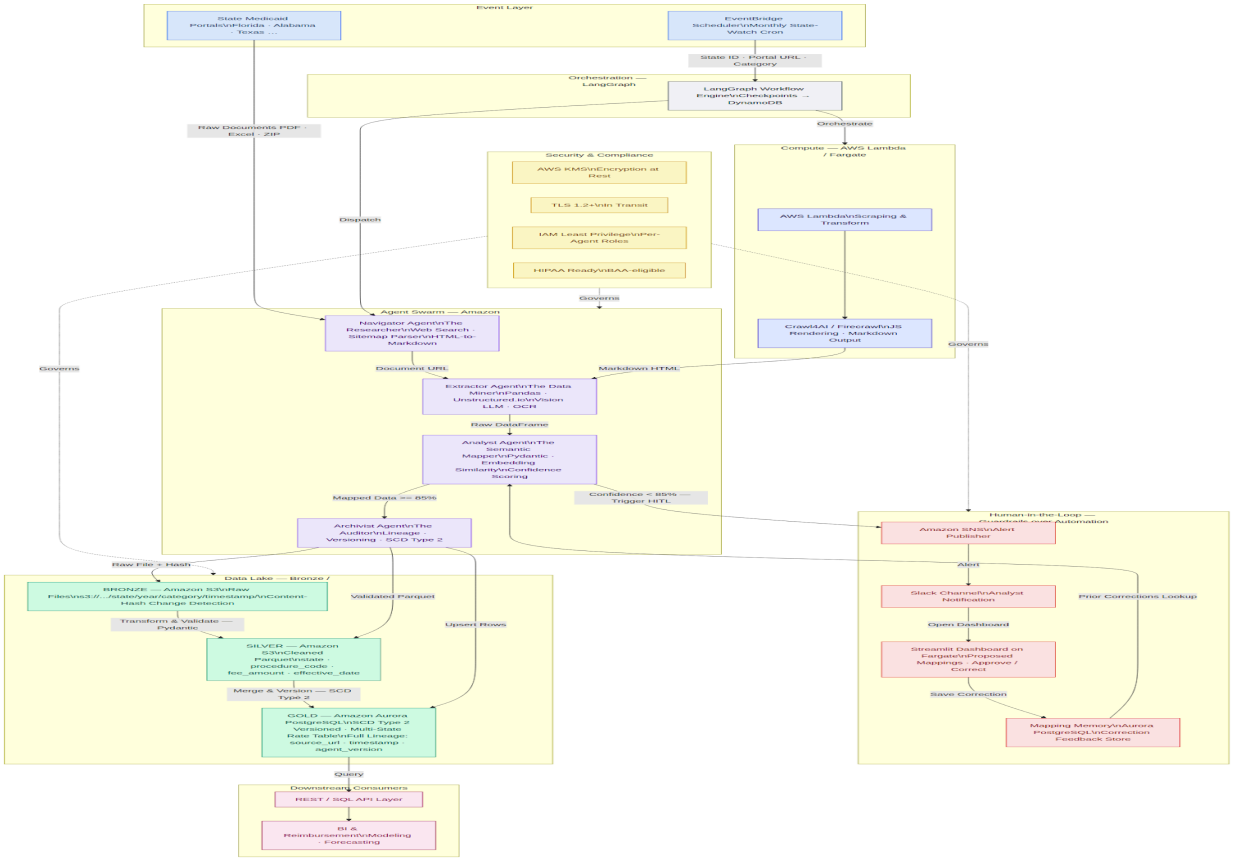

Figure 1: Sentinel-State System Architecture — *Color-coded zones show Event Layer (blue), Agent Swarm (purple), Compute (indigo), HITL (red), Data Lake (green), Security (yellow), and Consumers (pink). Arrows show all inter-component connections.*

How to Read Figure 1

Start at the top-left Event Layer: EventBridge fires the trigger. Follow the arrow right to LangGraph, which dispatches the Navigator Agent. Read the Agent Swarm left-to-right (Navigator → Extractor → Analyst → Archivist). Notice the red dashed arrow from the Analyst Agent downward — this is the HITL escalation path when confidence drops below 85%. The Data Lake (green) receives writes from the Archivist. The Security zone (yellow dashed lines) governs all three inner zones simultaneously.

2. Step-by-Step System Flow

The following walkthrough traces a single complete pipeline execution — from the EventBridge trigger to validated data available in the Gold API. Cross-reference each step with Figure 1 (architecture) and Figure 2 (sequence flow) as you read.

1. EventBridge fires a State-Watch Event. The scheduler passes three parameters: state identifier (e.g., FL), the portal root URL, and the dataset category (e.g., dental-fee-schedule). This event is published to LangGraph via an AWS event bridge rule configured per state/dataset combination.
2. LangGraph initializes a stateful workflow with a DynamoDB checkpoint. The Navigator Agent is dispatched with the portal URL. LangGraph records a checkpoint after each agent so the pipeline can resume from any point if a failure occurs downstream.
3. The Navigator Agent crawls the state portal. It attempts the previously recorded direct link first. If a 404 is returned, it parses the site sitemap and traverses menu hierarchies to locate the updated download link. The resolved URL is passed forward.
4. The Extractor Agent downloads the file. It detects the format (PDF, Excel, or ZIP), skips merged header rows, identifies relevant sheets in multi-tab workbooks, and performs OCR on scanned or image-based PDFs using the Vision LLM. A raw DataFrame with the original column names intact is passed to the Analyst.
5. The Analyst Agent performs semantic column mapping. For each raw column name, it computes an embedding similarity score against the canonical Medicaid schema fields and their known aliases. It first queries the Mapping Memory database to check if a human has previously approved or corrected this column for this state — if so, that decision is trusted unconditionally.
6. Confidence routing. If all column mappings score 85% or above, the mapped DataFrame is passed directly to the Archivist (automated path). If any mapping scores below 85%, the workflow pauses and Amazon SNS publishes an alert to the Slack analyst channel.
7. Human review (HITL path only). The analyst opens the Streamlit dashboard, reviews the proposed mappings, and either approves or corrects each one. The correction is saved to the Mapping Memory database and the workflow resumes.
8. The Archivist Agent writes to all three data lake layers: the original raw file to S3 Bronze (with SHA-256 content-hash metadata), the cleaned Parquet to S3 Silver, and the validated rows to Amazon Aurora Gold via a PostgreSQL UPSERT with SCD Type 2 versioning. Every row carries source_url, ingestion_timestamp, and agent_version for full audit lineage.
9. Downstream consumers query the Gold API layer via REST or SQL for reimbursement modeling, financial forecasting, and rate analytics across all states.

Runtime Sequence Flow

This diagram maps directly to the 9 steps above. Notice the alt/else branch at step 6 — this is the confidence routing decision.

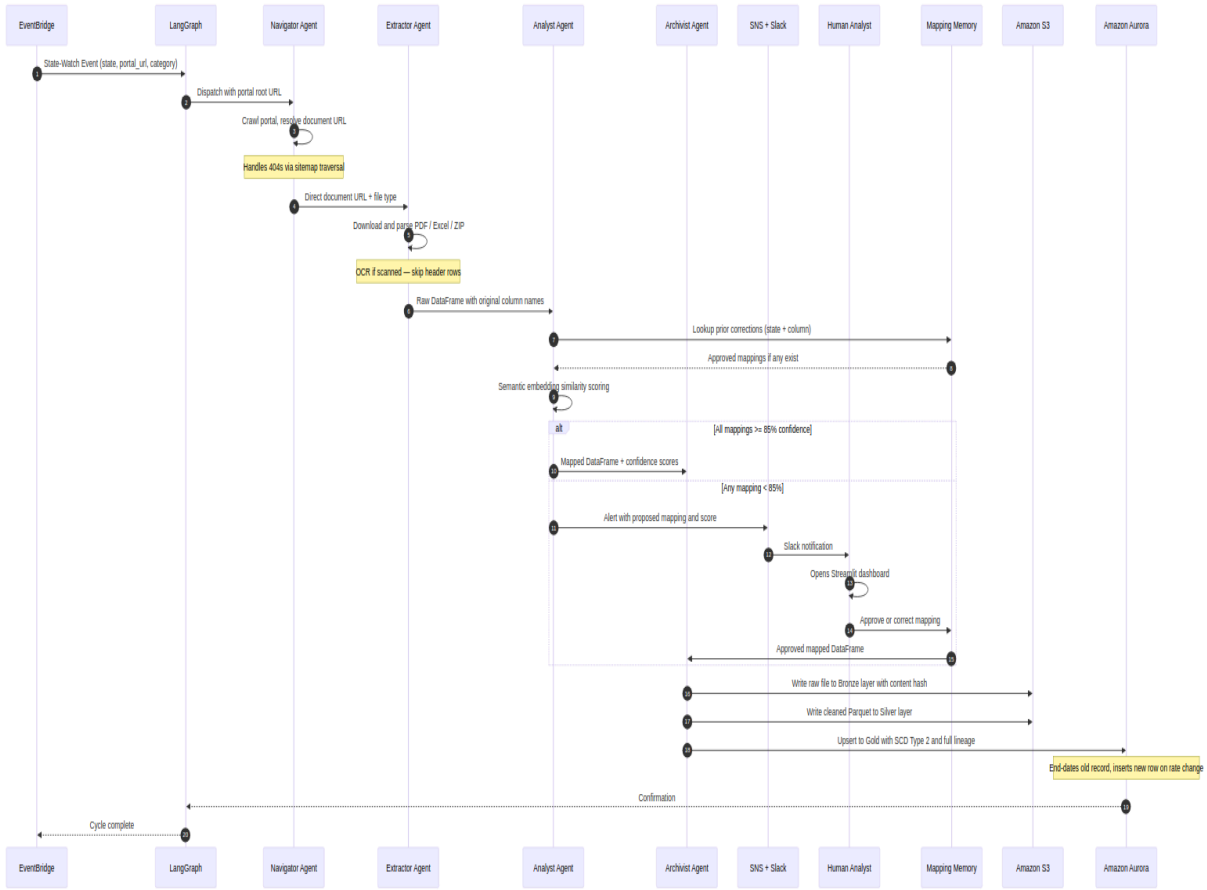

Figure 2: Sentinel-State Runtime Sequence Flow — *Numbered arrows correspond to the step-by-step flow in Section 2. The alt/else block shows the automated path (confidence >= 85%) vs the HITL escalation path (confidence < 85%). Dashed return arrows indicate async confirmation responses.*

How to Read Figure 2

Read top-to-bottom following the numbered arrows. The vertical dashed lines are participant lifelines. Solid arrows are requests; dashed return arrows are responses. The 'alt' block (with green and red sections) shows the conditional branch: green is the happy path (automated), red is the HITL escalation. Notice that Mapping Memory is consulted by the Analyst Agent at step 9 (before the confidence decision) and written to by the Streamlit UI at step 14 (after human correction).

3. Technical Stack

Sentinel-State is built exclusively on AWS-native services, supplemented by purpose-selected open-source libraries for AI orchestration, web scraping, and data validation. Every component was chosen for its ability to scale independently, operate serverlessly where possible, and integrate natively with IAM and KMS for security compliance.

Layer	Component	AWS Service	Purpose
Orchestration	LangGraph / Python	Lambda / Fargate	Stateful cyclic agent workflows with DynamoDB checkpoints
Intelligence	Amazon Bedrock	Claude 3.5 Sonnet	LLM reasoning and data extraction for all four agents
Compute	Serverless Execution	Lambda / Fargate	Scraping logic, data transformation, Streamlit dashboard
Scraping	Crawl4AI / Firecrawl	External (LLM-optimized)	LLM-optimized web crawling with JS rendering and Markdown output
Storage (Files)	Data Lake	Amazon S3	Bronze raw files, Silver Parquet, Gold aggregated datasets
Database	Relational + SCD Type 2	Amazon Aurora (PG)	Versioned rate history, Mapping Memory, Gold query surface
Validation	Pydantic Models	Python runtime	Strict schema enforcement and data type validation on Silver
HITL Interface	Review Dashboard	Streamlit on Fargate	Human review and approval of low-confidence column mappings
Notifications	Alert Channel	Amazon SNS + Slack	Sends < 85% confidence alerts to human analyst Slack channel

Why LangGraph over a Simple Pipeline

A linear function chain could orchestrate the four agents for simple cases, but Sentinel-State requires three capabilities that demand a proper graph framework: (1) stateful checkpointing so failed runs can resume mid-pipeline without re-downloading large files, (2) conditional branching for the HITL confidence threshold, and (3) the ability to insert human feedback back into the workflow mid-execution. LangGraph's cyclic graph model supports all three natively.

4. The Agent Swarm

Four specialized agents form the core processing pipeline. Each agent is a discrete LangGraph node with a defined input contract, internal reasoning loop powered by Amazon Bedrock, and a specific set of tools. Agents communicate exclusively through the LangGraph shared state object — there are no direct agent-to-agent calls, which ensures every state transition is logged and replayable.

Agent	Alias	Core Responsibility	Primary Input	Key Tools
Navigator	The Researcher	Finds the exact current file link on state portals; handles 404s by traversing site hierarchy and sitemaps	State portal root URL	Web Search, Sitemap Parser, HTML-to-Markdown
Extractor	The Data Miner	Downloads and parses PDFs, Excel, ZIPs; detects data start row; performs OCR on scanned documents	Document URL (PDF, XLS, ZIP)	Pandas, Unstructured.io, Vision LLM, OCR
Analyst	The Semantic Mapper	Maps raw column names to canonical schema fields using embedding similarity; scores confidence per mapping; queries Mapping Memory	Raw DataFrame from Extractor	Pydantic, Embedding Similarity, Mapping Memory DB
Archivist	The Auditor	Attaches lineage metadata (source_url, timestamp, agent_version) to every row; manages SCD Type 2 versioning	Mapped DataFrame + metadata	S3 API, Aurora PostgreSQL, SCD Type 2 logic

4.1 Navigator Agent — The Researcher

Input: State Portal Root URL | Output: Resolved Direct Document URL

The Navigator Agent is the entry point for every data collection cycle. It receives a state portal root URL and is responsible for finding the exact, current download link for the target dataset. Because state Medicaid portals frequently reorganize their content or move files, the Navigator must be resilient to broken links.

Navigation strategy:

- First attempt: Direct URL fetch from the previously recorded link.
- On 404: Parse the portal sitemap and traverse menu hierarchies to locate the updated link.
- Disambiguation: If multiple links match the category (e.g., multiple 'ACHN Participation Rates' pages), the agent uses recency signals and link text to select the most current document.
- Output: A validated, direct document URL passed to the Extractor Agent via LangGraph state.

Idempotency via Content-Hash

The Navigator compares the SHA-256 hash of the newly discovered file against the hash stored from the previous run. If hashes match, the file has not changed and the pipeline exits early without downstream processing. This prevents duplicate records in the Gold layer and reduces unnecessary LLM API calls.

4.2 Extractor Agent — The Data Miner

Input: Document URL (PDF / Excel / ZIP) | Output: Raw DataFrame

The Extractor Agent handles the full spectrum of file formats found across state Medicaid portals. Its primary challenge is that 'structured data' from government portals is rarely clean — PDFs have merged header rows, Excel files have multi-level column headers, and some files are scanned images with no extractable text layer.

Extraction strategy:

- PDF extraction: Uses Unstructured.io for text-based PDFs. Falls back to Vision LLM for scanned or image-based PDFs where OCR is required.
- Excel extraction: Uses Pandas to open multi-sheet workbooks. The agent identifies the correct sheet(s) using tab names and content heuristics, and detects where the actual data begins by skipping merged header rows.
- ZIP handling: Extracts the archive and recursively processes contained files.
- Output: A raw DataFrame with original column names preserved — no renaming occurs at this stage.

4.3 Analyst Agent — The Semantic Mapper

Input: Raw DataFrame | Output: Mapped DataFrame + Confidence Scores

The Analyst Agent is the most intelligence-intensive agent in the pipeline. Its task — translating heterogeneous raw column names into canonical schema fields — requires understanding not just the column name text, but the context provided by a sample of the column's values.

The Column Mapping Problem

A procedure code column might appear as: CPT_CD, Proc_Code, ProcedureID, PX_CODE, HCPCS_Code, SVC_CD, or simply Code. The Analyst Agent must recognize all of these as the canonical procedure_code field. It does this by embedding both the column name and a 5-row sample of values, then computing cosine similarity against a library of canonical field embeddings and their known aliases.

Mapping logic in order of precedence:

- Query Mapping Memory: if a human-approved mapping exists for (state, raw_column_name), use it unconditionally — no confidence check.
- Semantic embedding: embed (column_name + sample_values) and compare against canonical field embeddings. If score >= 85%, map automatically.
- New column detection: if a column has no strong match and no prior memory record, flag it as an unknown field — this always triggers HITL regardless of score.
- Low confidence: if any mapping scores < 85%, all unresolved mappings for the file are bundled into a single HITL alert (not one alert per column).

4.4 Archivist Agent — The Auditor

Input: Validated Mapped DataFrame + Metadata | Output: Persisted Gold Records

The Archivist Agent is the final step and the system's single writer. All persistence operations — Bronze, Silver, and Gold — occur in this agent, ensuring a clear single point of responsibility for data durability and lineage.

Persistence strategy:

- Bronze write: The original raw file is saved to S3 with the path pattern s3://sentinel-state-bronze/{state}/{year}/{category}/{timestamp}/raw_file. A content hash is recorded to detect file changes in future runs.
- Silver write: The cleaned, validated DataFrame is serialized to Parquet and written to S3 Silver.
- Gold merge: Records are upserted into Amazon Aurora using SCD Type 2 logic. When a rate changes, the old record is end-dated (end_date set) and a new record is inserted with the new rate and effective_date. This preserves the full rate history for any procedure code in any state.
- Lineage attachment: source_url, ingestion_timestamp, and agent_version are attached to every row before writing to Gold.

5. Data Processing Lifecycle

Data flows through three immutable persistence layers. Each layer is independently queryable for audit, recovery, and historical analysis. No layer overwrites another — Bronze is always the original source of truth, Silver is always the normalized version, and Gold is always the analytical surface.

Phase	Layer	Storage Path	Key Fields	Action
1 — Ingest	Bronze	s3://sentinel-state-bronze/{state}/{year}/{category}/{timestamp}/	Raw file as-is; SHA-256 content-hash tag	Download; no transform
2 — Transform	Silver	s3://sentinel-state-silver/{state}/{year}/{category}/data.parquet	state, procedure_code, modifier, fee_amount, effective_date, end_date	Pydantic validation; schema mapping
3 — Analyze	Gold	Amazon Aurora multi-state table (PostgreSQL)	All Silver fields + source_url + ingestion_timestamp + agent_version + SCD columns	Upsert; SCD Type 2 versioning

5.1 Bronze Layer — The Immutable Archive

Bronze is a write-once, read-many store. Files are stored exactly as downloaded, with no transformation applied. The Bronze layer serves two purposes: it provides a recovery path if downstream processing produces incorrect results, and it satisfies audit requirements by proving exactly what data was received from each state portal on each date.

- Retention: indefinite. No Bronze records are ever deleted.
- Change detection: content-hash comparison prevents reprocessing unchanged files.
- Format: original file format preserved (PDF, XLSX, CSV, etc.).
- Naming: path encodes state, year, category, and timestamp for easy time-range querying.

5.2 Silver Layer — The Validated Canonical Store

Silver is the transformation output. Every Silver file has been processed by the Extractor and Analyst agents, mapped to the canonical schema, and validated by Pydantic. Silver files are always in Parquet format for efficient column-oriented querying.

Canonical schema fields present in every Silver record:

- state — two-letter state code
- dataset_type — fee schedule category (e.g., dental, physician, pharmacy)
- procedure_code — HCPCS or CPT code
- modifier — optional procedure modifier
- fee_amount — reimbursement rate as a float
- effective_date — rate effective date
- end_date — rate end date (null if currently active)

5.3 Gold Layer — The Analytical Single Source of Truth

Gold is the query surface for all downstream consumers. It is a single multi-state PostgreSQL table in Amazon Aurora, enriched with SCD Type 2 version columns and full lineage metadata. Analysts never query Bronze or Silver directly — Gold is the only consumer-facing layer.

SCD Type 2 Versioning Explained

When the Archivist detects that a fee_amount for a given (state, procedure_code, modifier) has changed since the last run, it does NOT update the existing row. Instead: (1) the existing row's end_date is set to today, marking it as historical, and (2) a new row is inserted with the new fee_amount and a null end_date, marking it as the current active rate. This preserves the complete rate history — analysts can query 'what was the Florida Medicaid rate for CPT 97110 in Q2 2023' with a simple date-range filter.

6. Component Interaction Matrix

The following table is the authoritative reference for every significant interaction between system components. It documents the trigger condition, the exact payload exchanged, and the communication protocol. Use this table when debugging unexpected behavior or designing monitoring rules.

From	To	Trigger / Condition	Payload / Data Exchanged	Protocol

EventBridge	LangGraph	Scheduled cron (monthly per state)	State ID, portal root URL, dataset category	AWS Event
LangGraph	Navigator	Workflow start	Portal URL, state config, run ID	LangGraph dispatch
Navigator	Extractor	Document URL resolved	Direct document URL, file type, metadata	LangGraph state
Extractor	Analyst	Raw data extracted	DataFrame with original column names and values	LangGraph state
Analyst	Mapping Memory	Start of every mapping cycle	Lookup key: (state, raw_column_name)	PostgreSQL SELECT
Analyst	Archivist	All column confidence >= 85%	Mapped DataFrame + per-field confidence scores	LangGraph state
Analyst	SNS + Slack	Any mapping confidence < 85%	State, file name, raw column, proposed mapping, score	AWS SNS publish
Streamlit UI	Mapping Memory	Human approves or corrects	{raw_col, canonical_field, approved_by, timestamp}	PostgreSQL INSERT
Archivist	S3 Bronze	Always after download	Original raw file + SHA-256 content-hash metadata tag	S3 PutObject
Archivist	S3 Silver	After Pydantic validation pass	Cleaned Parquet file with canonical schema fields	S3 PutObject
Archivist	Aurora Gold	After Silver write	All fields + source_url + timestamp + agent_version	PostgreSQL UPSERT
All Agents	Amazon Bedrock	Any LLM reasoning task	Prompt + context; returns structured reasoning or extracted data	HTTPS API
BI / Analysts	Aurora Gold API	Ad-hoc or scheduled query	SQL queries; multi-state rate tables for modeling and forecasting	REST / SQL

7. Human-in-the-Loop (HITL) Workflow

The HITL mechanism is the platform's primary quality gate. It is intentionally minimalist — the system is designed to maximize automated throughput and engage humans only when confidence is insufficient. Over time, as the Mapping Memory accumulates human decisions, the frequency of HITL triggers decreases on a per-state basis.

#	Actor	Action	Output / Effect
1	Analyst Agent	Calculates semantic confidence score for each column mapping	Score per mapping (0-100%). If any < 85%, HITL is triggered for the entire file

2	Amazon SNS	Publishes structured alert event to the analyst Slack channel	Slack message: state name, file, all proposed mappings and their confidence scores
3	Human Analyst	Opens Streamlit dashboard link from Slack; reviews side-by-side proposed mappings	Decision per mapping row: Approve (accept suggestion) or Correct (select right canonical field)
4	Mapping Memory DB	Persists the human decision with full metadata	Durable correction record: used as trusted override in all future cycles for that state + column
5	Analyst Agent (next run)	Queries Mapping Memory before assigning any new mapping	Prior human approvals bypass the 85% threshold — fewer HITL triggers over time (self-improving)

7.1 Streamlit Review Dashboard

The Streamlit dashboard is deployed on AWS Fargate and is accessible via authenticated URL. It is a purpose-built, single-page review tool — not a general-purpose admin panel. Its design prioritizes fast decision-making: an analyst should be able to review and submit a typical file mapping in under 3 minutes.

Dashboard layout (left to right):

- Header row: state name, file name, ingestion timestamp, and total number of pending mappings.
- Mapping table: one row per column requiring review. Columns shown: Raw Column Name | Sample Values (5 rows) | Proposed Canonical Field | Confidence Score.
- Action controls: per-row Approve button (green) and Correct dropdown (full canonical field list).
- Submit All button: commits all decisions atomically and resumes the LangGraph workflow.

> **Why Streamlit on Fargate (not Lambda)**
> Streamlit requires a persistent server process — it maintains WebSocket connections for real-time UI updates. Lambda's stateless execution model is incompatible with this requirement. Fargate provides a long-running container that can be scaled to zero between review sessions to minimize cost.

7.2 Mapping Memory — The Learning Layer

Mapping Memory is more than a correction log — it is the mechanism by which Sentinel-State becomes progressively more autonomous over time. Every human decision is stored and reused in all future pipeline runs for the same state and column combination.

Schema: {state, raw_column_name, canonical_field, approved_by, approval_timestamp, correction_count, last_seen_value_sample}

- When the Analyst Agent processes a new file, it queries this table first using (state, raw_column_name) as the lookup key.

- A matching record with a prior human approval bypasses the confidence threshold check entirely — the mapping is trusted.
- Each time a mapping is corrected (not just approved), correction_count is incremented, providing an audit signal for columns that are persistently ambiguous.

8. Security & Compliance

Sentinel-State is designed to meet HIPAA technical safeguard requirements. While Medicaid fee schedules are not directly patient-identifiable, they are sensitive healthcare operational data that underpins reimbursement decisions. The platform applies defense-in-depth across every layer.

Control	Mechanism	AWS Service	Coverage
Encryption at Rest	AES-256	AWS KMS	All S3 buckets (Bronze, Silver, Gold) and Aurora instances
Encryption in Transit	TLS 1.2+	ACM / API Gateway	All inter-service and external portal communication
Access Control	Least Privilege IAM	AWS IAM	Each agent role scoped to specific S3 prefix and table only
Audit Trail	Full lineage chain	S3 + Aurora	Gold row traceable to Bronze file via source_url and hash
HIPAA Readiness	BAA-eligible services	AWS HIPAA services	Platform-wide; KMS + access logs + signed BAA
Schema Validation	Pydantic strict mode	Python (Lambda/Fargate)	Every incoming row at Silver transformation stage

8.1 IAM Role Architecture

Each agent runs under a dedicated IAM role with the minimum permissions required for its specific task. No agent can access another agent's resources, and no agent has delete permissions on any resource.

- Navigator Role: Read-only access to internet via NAT Gateway. No S3 or database permissions.
- Extractor Role: GetObject and PutObject on s3://sentinel-state-bronze/{state}/ prefix only.
- Analyst Role: Read access to S3 Silver prefix. Read/write to Mapping Memory table in Aurora.
- Archivist Role: PutObject on Silver prefix. Full write access to Aurora Gold table. Read access to Bronze for lineage resolution.

8.2 End-to-End Audit Trail

Every Gold-layer record carries a complete provenance chain. An auditor can reconstruct the exact source and processing history of any data point:

Audit Chain: Gold → Silver → Bronze → Source Portal

Gold row carries: source_url + bronze_path + ingestion_timestamp + agent_version.Bronze file at bronze_path is the exact original file from the portal.source_url is the direct portal download URL with timestamp.LangGraph checkpoint at agent_version contains the full agent reasoning trace for that run.This chain satisfies both internal audit requirements and potential regulatory review.

9. Deployment & Operations

9.1 EventBridge Schedule Design

Each (state, dataset_type) combination is configured as an independent EventBridge rule. This decoupled design means: a failure processing Florida dental rates does not block Alabama physician rates. Schedules are configurable per state — high-priority states can run weekly while stable states run monthly.

9.2 Failure Handling & Resumability

LangGraph checkpoints are written to DynamoDB after each agent node completes. If a downstream agent fails — for example, the Extractor crashes mid-download due to a transient network error — the pipeline resumes from the last successful checkpoint. The Navigator does not re-execute; the pipeline continues from the Extractor node with the already-resolved URL.

Retry Policy

Each agent node is configured with an exponential backoff retry policy: 3 retries with delays of 30s, 120s, and 300s. If all 3 retries fail, the LangGraph workflow transitions to a FAILED state, triggers a CloudWatch alarm, and sends a critical alert to the engineering on-call channel via SNS.

9.3 Idempotency

The Bronze layer content-hash mechanism ensures the pipeline is idempotent. Re-running the pipeline for the same state/date combination will detect that the file has not changed and exit early, preventing duplicate records in the Gold layer.

9.4 Monitoring Signals

- CloudWatch Metrics: agent execution duration, error rates, HITL trigger frequency per state.
- CloudWatch Alarms: alert on-call if any agent fails 3 consecutive runs for the same state.
- SNS Topics: separate topics for HITL alerts (routed to Slack analyst channel) and operational alerts (routed to on-call engineering channel).

10. Glossary

Term	Definition
Agent Swarm	The four specialized LangGraph nodes (Navigator, Extractor, Analyst, Archivist) that collectively process each state Medicaid file.
Amazon Bedrock	AWS-managed LLM service powering all four agents using Claude 3.5 Sonnet for reasoning and extraction tasks.
Bronze Layer	Raw, unmodified files downloaded from state portals. Stored in S3. No transformation applied. Retained indefinitely.
Canonical Schema	The standardized set of field names and data types that all ingested Medicaid data is normalized into during Silver transformation.
Confidence Score	A 0-100% score assigned by the Analyst Agent to each column mapping, reflecting semantic similarity between raw column name + sample values and the canonical field.
Content-Hash	SHA-256 hash of a downloaded file, stored as S3 metadata. Used by the Navigator to detect whether a file has changed since the last run.
Gold Layer	The analytical single source of truth. Amazon Aurora PostgreSQL. SCD Type 2 versioned. Full lineage. Queried by downstream BI consumers.
HITL	Human-in-the-Loop. Quality gate triggered when Analyst Agent confidence falls below 85%. Human reviews proposed mappings in the Streamlit dashboard.
LangGraph	Open-source Python framework for stateful, cyclic agent workflows with DynamoDB checkpointing. Orchestrates all four agents.
Mapping Memory	Aurora PostgreSQL table storing human-approved column-to-canonical-field corrections. Consulted by the Analyst Agent at the start of every run.
SCD Type 2	Slowly Changing Dimension Type 2. Versioning pattern that preserves history by end-dating old records and inserting new ones when values change.
Silver Layer	Cleaned, Pydantic-validated Parquet files on S3. Canonical schema applied. Intermediate layer between Bronze and Gold.
Streamlit Dashboard	Fargate-hosted web UI for human review of low-confidence column mappings. Single-page; optimized for fast approval decisions.
State-Watch Cycle	One complete execution of the pipeline for a single (state, dataset_type) combination, triggered by EventBridge on a configured schedule.

End of Document — Sentinel-State Enterprise Architecture v2.0 — Includes Figures 1 & 2